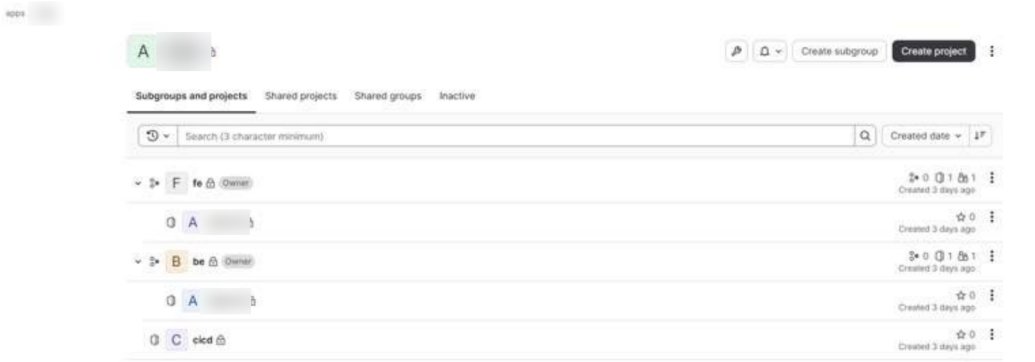
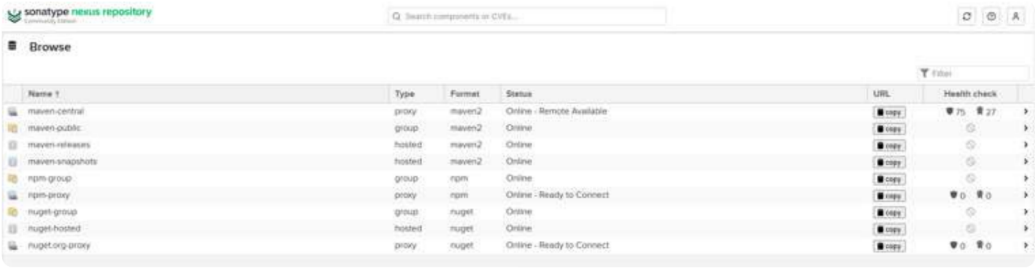


Gitlab



Nexus

개요

사내 Jenkinsfile 템플릿을 사용해 BE와 FE를 [redacted] 클러스터에 CI/CD로 배포했고, 수동 실행, GitLab push 트리거 설정, 빈 커밋 테스트까지 모두 끝냈다. Gitlab에 push하면 빌드, 이미지 push, 매니페스트 갱신, Argo CD 배포가 자동으로 이어진다.

과제 순서

1. BE/FE 파이프라인을 수동 실행해 배포와 동작을 확인한다. (완료)
2. BE/FE Jenkins Job에 GitLab push 트리거를 설정한다. (완료)
3. 소스 변경 없이 빈 커밋을 push해 자동 배포를 확인한다. (완료)

배포 대상

구분	내용
BE	Spring Boot, PostgreSQL 사용
FE	Vite/React, nginx로 서빙
DB	같은 네임스페이스의 Pod(서비스 `task-db`), 테이블과 데이터는 BE가 기동할 때 생성
네임스페이스	[redacted]
Argo CD 프로젝트	[redacted]

Argo CD 앱	`[redacted]-be-task-be`, `[redacted]-fe-task-fe`
매니페스트 저장소	`apps/[redacted]/cicd`, 브랜치 `dev`, 경로 `<job>/cicd/k8s`

사용 환경

구성 요소	주소	용도
GitLab	`gitlab-[redacted]`	소스와 매니페스트 저장소
Jenkins	`jenkins-[redacted]`	빌드와 파이프라인 실행
Nexus	`nexus-[redacted]`	사내 의존성 저장소
Harbor	`harbor-[redacted]`	컨테이너 이미지 저장소
Argo CD	`argocd-[redacted]`	매니페스트 기반 배포
서비스(FE)	`task-[redacted]`	배포 결과 접속. DNS 미등록으로 PC hosts를 임시 사용

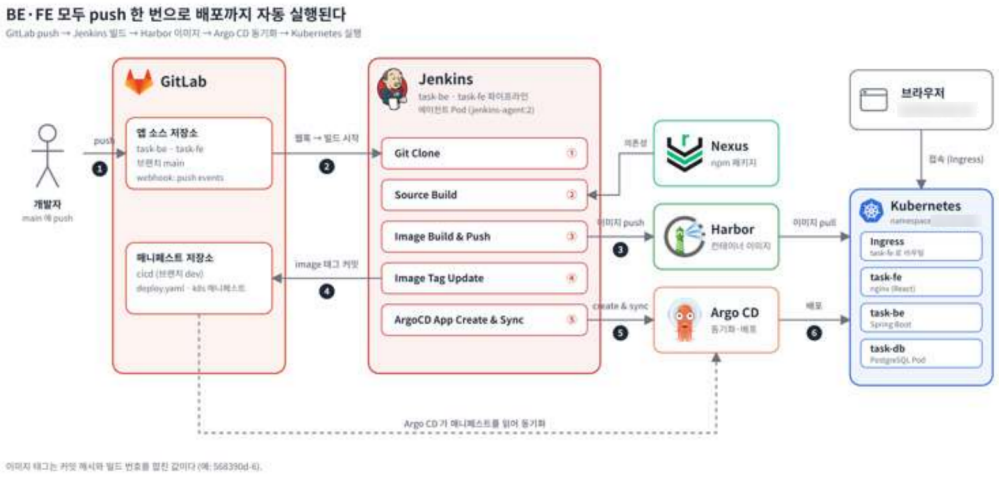
추가사항

항목	처리
사내 개발계 DB에 영향X	`DB_URL`이 같은 네임스페이스의 `task-db`를 가리키는 것을 확인했다
BE/FE 소스는 수정X	소스는 읽기만 했다. 설정은 Job 파라미터와 매니페스트로만 처리했다
모든 계정은 admin을 사용	멘토 지시대로 적용했다
`secret.yaml`은 Git에 보관	유지했다.
Jenkinsfile은 사내 템플릿을 최소 변경	`agent any`, `tools {}` 구조를 유지했다

공용 인프라 변경(values엔 적용X)	Harbor Ingress 본문 크기(client-max-body-size="0")와 전역 `http2` 적용

- Harbor에 에이전트 이미지를 업로드하기 위해 크기 제한을 풀었다.
- F5 NGINX Ingress는 http2가 기본값 False → argoCD login이 계속 실패했다.
  - `[redacted] ingress` 네임스페이스의 `nginx-ingress` ConfigMap에 `http2: "true"`를 직접 넣었다.
  - Argo Ingress에만 `nginx.org/http2`를 붙여 봤지만 효과가 없었다.

진행 흐름



환경과 파이프라인을 만들고 BE, FE 순서로 배포를 확인했다.

환경 구성

Jenkins 컨트롤러(2.580)는 에이전트 JVM이 Java 21 이상이어야 하고 BE 빌드는 JDK 17을 써야 한다. 그래서 BE와 FE를 함께 빌드할 수 있는 에이전트 이미지를 직접 만들어 Harbor에 올렸다.

에이전트 이미지

이미지: `[redacted]/library/[redacted]/jenkins-agent:2` (압축 약 656MB)

구성	경로 / 버전	용도
베이스	`quay.io/buildah/stable` (Fedora)	buildah로 이미지 빌드
JDK 25	`/opt/java/jdk25` (기본 java)	Jenkins 에이전트 실행
JDK 17	`/opt/java/openjdk`	BE 빌드 (Tools `jdk-17`)

Maven	`/opt/maven`	BE 빌드 (Tools `maven-3.9.9`)
Node 24	`/opt/node`	FE 빌드 (Tools `node-24.15.0`)
argocd CLI	v3.5.5	Argo CD 단계
그 외	git, procps, tar 등	파이프라인 실행

이미지 안에서 buildah를 돌리기 위한 환경 변수다.

```
ENV BUILDAH_ISOLATION=chroot STORAGE_DRIVER=vfs
ENTRYPOINT ["/usr/local/bin/jenkins-agent"]
```

클러스터 노드가 불안정해 이미지는 클러스터가 아니라 Windows PC의 Docker Desktop에서 `linux/amd64` 로 빌드하고 push했다.

이미지 빌드를 jenkins에서 수행하려고 했으나 메모리 부족으로 워커 노드가 notready됐다.

```
docker build --platform linux/amd64 -t [redacted]/library/[redacted]/jenkins-agent:2 -f jenkins-agent.v2.Dockerfile .
docker push [redacted]/library/[redacted]/jenkins-agent:2
```

Jenkins Pod 템플릿

라벨을 비워 둔 `default` 템플릿 하나로 만들어 `agent any` 가 이 템플릿을 쓰게 했다.

항목	값
이름 / 네임스페이스	`default` / `jenkins` (라벨 없음)
컨테이너	`jnlp`, 이미지 `jenkins-agent:2`, **privileged**, root(0/0)
requests	CPU 200m, 메모리 512Mi, 임시 스토리지 1Gi
limits	CPU 2, 메모리 3Gi, 임시 스토리지 10Gi
볼륨	emptyDir `/var/lib/containers` (buildah 저장소)
환경 변수	`LANG=C.UTF-8`, `MAVEN_OPTS=-Xmx1g`, `NODE_OPTIONS=--max-old-space-size=1536`
실행 옵션	동시성 2, Pod Retention Never, 연결 타임아웃 600초
Built-In Node	executors 0 (컨트롤러에서 빌드하지 않음)

클러스터 제약

항목	내용	대응
워커 노드	1대 (12 CPU, 약 27GiB). 메모리 requests 82%, limits 103%	**빌드 파드에 메모리 limit**을 건다. 무거운 빌드는 클러스터 밖에서 한다
Ingress 컨트롤러	F5 NGINX Ingress 5.6.3, IngressClass `nginx`	`nginx.org/*` 어노테이션만 인식한다
스토리지	StorageClass `nfs-csi`	DB에 필요하면 PVC를 쓴다

파이프라인 구성

사내 템플릿의 구조(`agent any`, `tools {}`)를 그대로 두고 값만 채웠다.

Job 구성

항목	값
Job 이름	`task-be`, `task-fe`
위치	폴더 `apps/[redacted]/dev`
유형	Pipeline (스크립트를 Job에 직접 입력)
자격 증명 ID	`GITLAB`, `NEXUS`, `HARBOR`, `ARGOCD`
파라미터	String 파라미터

공통 `environment` 블록이다. 이미지 경로와 Argo 앱 이름이 여기서 결정된다.

```
environment {
    COMPONENT_TYPE  = "be"   // FE는 "fe"
    DEPLOY_APP_NAME = "${JOB_BASE_NAME}"
    DEPLOY_PATH     = "${DEPLOY_APP_NAME}/cicd/k8s"
    IMAGE_REPO_URL  = "[redacted]/library"
    IMAGE_BASE_NAME =
"${IMAGE_REPO_URL}/${NS}/${NS}-${COMPONENT_TYPE}-${DEPLOY_APP_NAME}"
    ARGOCD_URL      = "[redacted]:443"
    ARGOCD_APP_NAME = "${NS}-${COMPONENT_TYPE}-${DEPLOY_APP_NAME}"
}
```

단계별 동작

단계	BE	FE

Git Clone	소스 clone 후 이미지 태그를 `<커밋 해시>-<BUILD_NUMBER>`로 정한다	같음
Source Build	`mvn ${BUILD_GOAL:-clean package} -DskipTests=true -s "$MAVEN_SETTINGS_XML"`	`npm ci`(lock 없으면 `npm install`) 후 `npm run build`, `dist/index.html` 확인
Image Build & Push	Dockerfile을 단계 안에서 생성, JRE 이미지에 jar 복사, buildah로 bud/push	nginx 이미지에 `COPY dist/ /usr/local/nginx/`, buildah로 bud/push
Image Tag Update	매니페스트 저장소(`dev`)를 clone하고 `deploy.yaml`의 `image:` 줄을 새 태그로 바꾼 뒤 commit/push	같음
ArgoCD App Create & Sync	CLI로 로그인, 프로젝트/저장소/앱 upsert, sync	같음

이미지 태그를 바꾸는 명령이다. `deploy.yaml`의 모든 `image:` 줄이 바뀌기 때문에 DB 매니페스트는 다른 파일에 둔다.

```
sed -i -E 's%image:.+%image: ${IMAGE_NAME}%' ${DEPLOY_PATH}/deploy.yaml
```

Job 파라미터

파라미터	BE	FE
`NS`	[redacted]	[redacted]
`SOURCE_GIT_URL`	`gitlab-[redacted]/apps/[redacted]/be/[redacted]-be.git`	`gitlab-[redacted]/apps/[redacted]/fe/[redacted]-fe.git`
`SOURCE_BRANCH`	`main`	`main`

`CICD_GIT_URL`	`gitlab-[redacted]/apps/[redacted]/cicd.git`	`gitlab-[redacted]/apps/[redacted]/cicd.git`
`CICD_BRANCH`	`dev`	`dev`
`BUILD_GOAL`	비움	비움
`BUILD_RESULT`	`target/[redacted]-0.0.1-SNAPSHOT.jar`	`build`
`DOCKER_BASE_IMAGE`	`docker.io/eclipse-temurin:17-jre-noble`	`docker.io/nginx:1.31.6-alpine3.24-slim`
`DOCKER_DEPLOY_PORT`	`8080`	`80`
`DOCKER_DEPLOY_PROFILE`	`default`	없음

Argo CD 단계

[redacted]와 같은 CLI 방식이다. [redacted] Jenkinsfile을 참고해 작은따옴표 `sh '''`를 써서 Groovy 보간 경고를 피했고 repo 등록 줄을 추가했다.

```
argocd login "${ARGOCD_URL}" --username "${ARGOCD_USER}" --password "${ARGOCD_PSW}" --grpc-web
argocd proj create "${NS}" --src='*' --dest='*',"${NS}" --upsert --grpc-web
argocd proj allow-cluster-resource "${NS}" '*' '*' --grpc-web
argocd repo add "https://${CICD_GIT_URL}" --username "${GIT_USER}" --password "${GIT_TOKEN}" --upsert --grpc-web || true
argocd app create "${ARGOCD_APP_NAME}" --project "${NS}" \
    --repo "https://${CICD_GIT_URL}" --revision "${CICD_BRANCH}" --path "${DEPLOY_PATH}" \
    --dest-name in-cluster --sync-option CreateNamespace=true --dest-namespace "${NS}" --upsert --grpc-web
argocd app sync "${ARGOCD_APP_NAME}" --grpc-web
```

매니페스트 구성

대상	내용
BE	ConfigMap(`DB_URL=jdbc:postgresql://task-db:5432/taskdb`, `DB_USERNAME`), Secret, Service, Deployment, egress NetworkPolicy(사내 영향 줄이기 위함)
DB	`task-db` 서비스를 가진 PostgreSQL Pod. 별도 매니페스트 파일로 둔다

FE	nginx ConfigMap을 `/etc/nginx/conf.d`에 마운트한다. `root /usr/local/nginx`, `/api/`는 `task-be:8080`으로 프록시, SPA는 `try_files`로 처리한다
Ingress	호스트 `task-`[redacted], TLS Secret [redacted] `tls`
브랜치 규칙	매니페스트는 `dev`에만, cicd 스크립트 파일은 `main`에만 push한다

F5 NGINX Ingress에 맞춰 FE Ingress를 다음과 같이 고쳤다. 이전 `nginx.ingress.kubernetes.io/*` 어노테이션은 제거했다.**(파일 첨부 기본값 방지)**

```
metadata:
  annotations:
    nginx.org/client-max-body-size: "50m"
    nginx.org/proxy-read-timeout: "300s"
    nginx.org/proxy-send-timeout: "300s"
spec:
  ingressClassName: nginx
```

배포 결과

BE, DB, FE가 모두 올라왔고, 브라우저에서 관리자로 로그인해 접수 목록을 조회하는 것까지 확인했다.

확인 항목	방법	결과
BE 파이프라인	Jenkins 빌드 콘솔	Argo 단계까지 통과, `Finished: SUCCESS`
매니페스트 커밋	`dev` 브랜치 커밋 내용	`image:` 줄 하나만 변경된 것을 확인
Argo CD 동기화	`argocd app sync` 출력	`Synced to dev`, `successfully synced`
BE Pod	`kubectl get pods`	`Running`, DB 접속 성공
DB 접속 대상	`DB_URL` 확인	같은 네임스페이스의 `task-db`
FE 파이프라인	Jenkins 빌드 콘솔	성공
FE 접속	`https://task-`[redacted]	화면 표시
로그인·API	관리자 로그인 후 접수 목록 조회	로그인 성공, `/api` 프록시와 DB 조회 동작

BE 파이프라인의 Argo 단계 로그

```
'admin:login' logged in successfully
application '[redacted]-be-task-be' created
Sync Status:        Synced to dev (a0bfd69)
Phase:              Succeeded
Message:            successfully synced (all tasks run)
```

배포 후 확인

```
kubectl -n [redacted] get pods,svc,ingress
kubectl -n [redacted] logs deploy/task-be --tail=50
kubectl -n [redacted] describe ingress task-fe
```

자동 배포 확인 (2~3단계)

BE와 FE 소스 저장소에 GitLab 웹훅을 설정하고 빈 커밋을 push해 자동 빌드와 배포가 이어지는 것을 확인했다. 소스 파일은 바꾸지 않았다.

확인 항목	방법	결과
웹훅	GitLab Webhooks의 Test → Push events	요청이 Jenkins에 도달해 빌드가 자동으로 시작
빈 커밋 push	`git commit --allow-empty` 후 `main`에 push	`Started by GitLab push`로 빌드가 자동 시작
이미지 태그	GitLab 커밋과 Harbor 태그 비교	커밋 해시가 일치한다
매니페스트·배포	cicd `dev` 커밋과 파드 확인	이미지 태그 갱신과 Argo CD 배포까지 정상 동
동시 실행	Job 설정	`Do not allow concurrent builds`를 켜고 `Abort previous builds`는 끈다. 연속 push는 큐에서 차례로 실행된다

이슈와 조치

진행 중 발생한 문제는 19건이고 모두 원인을 찾아 조치했다. 클러스터 안정성(노드, GitLab), 이미지·네트워크(Harbor, Ingress, Argo), Jenkins 환경, 앱 동작 순으로 정리했다.

클러스터 안정성

번호	현상	원인	조치
1	에이전트 이미지 빌드 후 워커 노드가	클러스터 안 빌드가 원인이라고 추측했고 VM	이미지를 PC의 Docker Desktop

	NotReady가 되었다	멈춤이 메모리 문제인 것을 확인했다.	에서 빌드했다. 빌드 파드에 메모리 limit(3Gi)을 건다. 클러스터 안의 무거운 빌드를 피한다
2	GitLab이 502를 반환하고 `gitlab-0`이 재시작을 반복했다	PostgreSQL의 빈 lock 파일 `/var/opt/gitlab/postgresql/.s.PGSQL.5432.lock`이 남아 기동에 실패했다. 초반에 probe 시간이 충분한지 먼저 확인했다.	lock 파일을 삭제하자 정상 기동했다

이미지·네트워크

번호	현상	원인	조치
3	Harbor push가 `413 Request Entity Too Large`로 실패했다	클러스터 Ingress는 F5 NGINX Ingress라 `nginx.ingress.kubernetes.io/*`를 무시하고, 기본 `client-max-body-size`가 1m이다. 처음에 커뮤니티 컨트롤러의 어노테이션이라고 추측했는데 틀렸다	`harbor-ingress`에 `nginx.org/client-max-body-size: "0"`을 적용했다
4	`argocd login --grpc-web`이 `gRPC connection not ready`로 실패했다	F5 NGINX Ingress의 `http2` 설정이 기본 False라 gRPC-Web 연결이 안 된다. Argo Ingress에 개별 `nginx.org/http2`를 붙여도 효과가 없었다	[illegible] `ingress/nginx-ingress` ConfigMap에 `http2: "true"`를 전역으로 켰다. 이후 curl이 `http=2`, CLI 로그인이 성공했다.

5	FE Ingress가 반영되지 않고 브라우저에 `ERR_SSL_UNRECOGNIZED_NAME_ALERT`가 나왔다	파이프라인을 돌리기 전이라 FE Ingress가 아직 배포되지 않았다. 또한 매니페스트의 `ingressClassName: nginx-internal`이 클러스터에 없다(클러스터의 IngressClass는 `nginx` 하나). 배포하더라도 어떤 컨트롤러도 처리하지 않는 상태였다	Ingress를 `nginx` 클래스와 `nginx.org/*` 어노테이션으로 고쳐 `dev`에 push한 뒤 파이프라인을 실행했다. 이후 접속이 되었다

Jenkins 환경

번호	현상	원인	조치
6	테스트 파이프라인이 컨트롤러(`jenkins-0`)에서 돌아 `mvn: not found`가 나왔다	Built-In Node에 executor가 있어 `agent any`가 컨트롤러에서 실행된다	Built-In Node executors를 0으로 설정했다
7	에이전트가 `UnsupportedClassVersionError` (class 65 vs 61)로 기동하지 못한다	Jenkins 2.580 컨트롤러는 에이전트 JVM이 Java 21 이상이어야 한다. 에이전트 JVM과 빌드용 JDK를 같은 것으로 봤다	이미지 v2에 JDK 25(에이전트)와 JDK 17(Tools `jdk-17`)을 분리해 넣었다
8	`agent any` 빌드가 "Waiting for next available executor"에 머물렀다	라벨 없는 Pod 템플릿이 없었다	`default` Pod 템플릿을 만들었다
9	테스트 스크립트에서 `hostname: command not found`가 나왔다	이미지에 `hostname` 명령이 없다	`uname -n`으로 대체했다
10	Source Build가 빌드를 마친 뒤 이미지	Job 파라미터 `BUILD_RESULT`가	[illegible]와 같은 Dockerfile로 맞

	단계에서 `COPY build ...`로 실패했다 (FE)	`build`였는데 Vite는 `dist`에 출력한다. [illegible]는 `COPY dist/ /usr/local/nginx/`로 고정한다	쳤다. nginx 설정은 매니페스트 ConfigMap의 `root /usr/local/nginx`를 따른다

앱 동작과 보안

번호	현상	원인	조치
11	Argo 단계에서 Groovy 보간으로 비밀번호가 노출된다는 경고가 나왔다	큰따옴표 `sh """`에 자격 증명 변수를 써서다	[illegible]와 같은 `sh '''`로 바꾸어 변수를 쉘이 해석하게 했다

후속 조치와 요청 사항

자동 트리거 설정 요약

항목	설정
Jenkins 트리거	Build when a change is pushed to GitLab, Push Events만 사용한다
허용 브랜치	소스 배포 브랜치 `main`만
Secret token	Job마다 생성해 GitLab 웹훅에 등록했다.
GitLab 웹훅	소스 repo마다 하나. Push events, 브랜치 `main` (Wildcard pattern), URL은 해당 Job의 `/project/...` 주소
동시 실행	Do not allow concurrent builds를 켠다.

웹훅 URL 형태다. Job이 폴더 안에 있어 폴더 경로가 포함된다.

```
https://[illegible]/project/apps/[illegible]/dev/task-be
https://[illegible]/project/apps/[illegible]/dev/task-fe
```

빈 커밋 테스트 명령

```
git commit --allow-empty -m "trigger test"
git push origin main
```

보고

아래 항목은 운영 설정을 직접 바꾼 것이라 Helm이나 JCasC가 다시 적용하면 사라질 수 있다.

항목	현재 상태
전역 `http2`	`[illegible] ingress/nginx-ingress` ConfigMap을 직접 수정했다
Harbor Ingress 본문 크기	`nginx.org/client-max-body-size: "0"`을 어노테이션으로 적용했다
Jenkins Pod 템플릿과 Tools	UI에서 생성했다
`jenkins` 네임스페이스 라벨	buildah를 위해 `pod-security.kubernetes.io/enforce=privileged`를 설정했다
장애 이력	노드 NotReady, GitLab·Harbor DB 재시작 반복이 있었다